Computational thinking: abstraction and decomposition

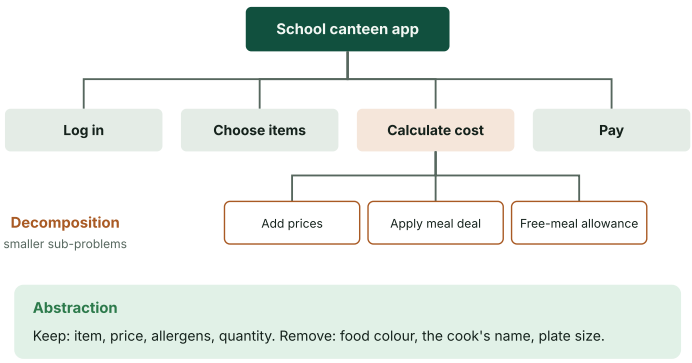
D09 FACT FILE

Specification: students should be able to

- Explain in simple terms the underlying concepts of computational thinking – abstraction and decomposition

CCEA fact file: Digital Design Principles

Key words: computational thinking, abstraction, decomposition, sub-problem, top-down design, structure diagram, algorithm



Decomposition of a school canteen app into sub-problems, with an abstraction note

Abstraction and decomposition

Abstraction means removing details that do not matter so the problem becomes simpler – a model keeps only what is needed. Decomposition means breaking a problem down into smaller sub-problems that are easier to understand, solve and test individually; a structure diagram shows this top-down. Together they let a programmer design an algorithm, a precise sequence of steps that solves the problem.

Benefits

Abstraction reduces complexity and the amount of data and code required. Decomposition makes large problems manageable, lets team members work on separate parts at the same time, allows parts to be reused, and makes testing and debugging easier because each module can be checked separately.

T Key terms from the mark schemes

Top-down design
Breaking the main task into smaller sub-tasks.
Credited: U4 2025 Q7

Decomposition
Breaking a complex problem into smaller, more manageable sub-problems.
Credited: U4 2019 Q6, 2022 Q1, 2023 Q4

Abstraction
Removing details of a problem that are not needed for the solution.
Credited: U4 2019 Q6, 2022 Q1, 2023 Q4

Exam focus

2019 Q6 (QWC), 2022 Q1(d)–(e) (definitions) and 2023 Q4(a) (fill the gaps in a controlled assessment scenario).

Unit 4 2023 Q4	[23]	Decomposition and abstraction; flowchart symbols; syntax/logic errors; functions; menu validation loop
Unit 4 2022 Q1(d)–(g)	[4]	Abstraction, decomposition, search efficiency, ASCII
Unit 4 2019 Q6	[6]	Abstraction and decomposition (levelled)

? Check yourself

1. Define abstraction. [2]
2. Define decomposition. [2]
3. A cinema wants a booking program. Give two details that could be removed through abstraction. [2]

Answers: 1. Removing or hiding details that are not needed to solve the problem (1) / so you focus only on what is important and the problem becomes simpler (1) 2. Breaking a large, complex problem down (1) / into smaller, more manageable sub-problems that can each be solved (and tested) separately (1) 3. Any two relevant details, e.g.: the customer's favourite colour (1) / the colour of the seats/carpet (1) / the plot or actors of the film (1) / the customer's hair colour or clothing (1) / the cinema building's address history (1). Accept any detail not needed for a booking; do not accept film, time, seat, number of tickets or price.